Complete C++ STL in 1 Video

Lecture notes with time complexity cheat sheets, examples, and interview-focused tips

Video: Complete C++ STL in 1 Video | Time Complexity and Notes

Focus: C++ Standard Template Library containers, iterators, algorithms, comparators, and practical DSA usage.

Best use: Read once for concepts, then keep the complexity tables nearby while solving problems.

Quick promise: STL is not just syntax. It is a toolbox of ready-made data structures and algorithms. The main skill is choosing the right container and knowing the cost of each operation.

Prepared as structured lecture notes from the linked lecture topic. If you want, you can later ask for a shorter revision sheet or flashcards from these notes.

1. What is STL?

The C++ Standard Template Library, or STL, is a collection of generic containers, algorithms, iterators, and function objects. It helps you write concise and efficient code for data-structure-and-algorithm problems.

- Containers: ready-made data structures such as `vector`, `set`, `map`, `queue`, and `priority_queue`.
- Algorithms: reusable operations such as `sort`, `reverse`, `binary_search`, `lower_bound`, and `next_permutation`.
- Iterators: pointer-like objects that connect containers with algorithms.
- Comparators: custom ordering rules for sorting, sets, maps, and priority queues.

Common headers

```
#include <bits/stdc++.h>   // common in competitive programming
using namespace std;

// Standard alternatives:
#include <vector>
#include <set>
#include <map>
#include <queue>
#include <algorithm>
```

Exam/interview advice: STL can save implementation time, but you must still know what data structure is being used internally and the expected time complexity.

2. Pairs and basic utility types

A `pair` stores two values together. It is useful for coordinates, key-value style grouping, graph edges, and sorting by multiple fields.

```
pair<int, int> p = {3, 5};
cout << p.first << " " << p.second;

pair<int, pair<int,int>> x = {1, {2, 3}};
cout << x.second.first;   // 2
```

Vector of pairs

```
vector<pair<int,int>> intervals;
intervals.push_back({1, 4});
intervals.push_back({2, 8});
sort(intervals.begin(), intervals.end());  // sorts by first, then second
```

3. Iterators

Iterators behave like pointers to elements inside containers. Most STL algorithms accept a half-open range: `[begin, end)`. This means the start is included and the end is excluded.

- `v.begin()`: iterator to the first element.
- `v.end()`: iterator just after the last element; do not dereference it.
- `rbegin()` and `rend()`: reverse iteration.

- Use `auto` for long iterator types.

```
vector<int> v = {10, 20, 30};
for (auto it = v.begin(); it != v.end(); ++it) {
    cout << *it << " ";
}

for (auto x : v) cout << x << " ";
```

Important: Some operations invalidate iterators. For example, inserting into a vector can reallocate memory and make old iterators invalid.

4. Container map: what to use when

Need	Prefer	Why
Dynamic array, random access	vector	Fast indexing; cache-friendly; amortized O(1) push_back
Insert/delete frequently at both ends	deque	O(1) front/back operations
LIFO order	stack	Top, push, pop only
FIFO order	queue	Front, push, pop only
Need min/max repeatedly	priority_queue	Heap; top O(1), push/pop O(log n)
Sorted unique elements	set	Balanced BST; log n operations
Sorted key-value mapping	map	Balanced BST; keys sorted
Fast average lookup	unordered_set/map	Hash table; average O(1), worst O(n)
Count duplicates	multiset / unordered_multiset	Allows repeated values

Rule of thumb: if order matters, use ordered containers like `set` or `map`. If only lookup speed matters, use unordered containers, but remember worst-case hash collisions.

5. vector

`vector` is the most commonly used STL container. It is a dynamic array: elements are stored contiguously and can be accessed by index.

```
vector<int> v;
v.push_back(10);
v.emplace_back(20);
cout << v[0];          // 10
v.pop_back();
cout << v.size();

vector<int> a(5, 100); // five elements, all 100
```

Operation	Time	Notes
v[i]	O(1)	Random access
push_back / emplace_back	Amortized O(1)	May become O(n) during reallocation
pop_back	O(1)	Removes last element
insert / erase in middle	O(n)	Shifts elements
size	O(1)	Number of elements
clear	O(n)	Destroys elements

Useful vector tricks

- `sort(v.begin(), v.end())` sorts in ascending order.
- `sort(v.rbegin(), v.rend())` sorts in descending order.
- `v.erase(unique(v.begin(), v.end()), v.end())` removes consecutive duplicates after sorting.
- `reserve(n)` can reduce reallocations when the final size is known.

6. deque, list, stack, and queue

deque

`deque` is a double-ended queue with efficient insertion and deletion at both front and back.

```
deque<int> dq;
dq.push_front(5);
dq.push_back(10);
dq.pop_front();
dq.pop_back();
```

list

`list` is a doubly linked list. It supports efficient insertion/deletion when you already have the iterator, but it does not support random indexing. In most DSA problems, `vector` is preferred unless frequent middle insertion/deletion is essential.

stack and queue

```
stack<int> st;
st.push(1); st.push(2);
```

```
cout << st.top();
st.pop();

queue<int> q;
q.push(1); q.push(2);
cout << q.front();
q.pop();
```

Container	Access	push	pop	Typical use
stack	top O(1)	O(1)	O(1)	DFS, parentheses, monotonic stack
queue	front/back O(1)	O(1)	O(1)	BFS, scheduling
deque	front/back/index O(1)	O(1) ends	O(1) ends	Sliding window, two-ended processing

7. priority_queue

`priority_queue` is a heap-based container adaptor. By default, C++ gives a max-heap, so the largest element is at the top.

```
priority_queue<int> pq;      // max heap
pq.push(10); pq.push(4); pq.push(20);
cout << pq.top();          // 20
pq.pop();

priority_queue<int, vector<int>, greater<int>> minpq; // min heap
```

Operation	Time	Notes
top	O(1)	Read highest-priority element
push / emplace	O(log n)	Insert and heapify
pop	O(log n)	Remove top and heapify
empty / size	O(1)	Metadata operations

Custom priority for pairs

```
// Min-heap by first value
priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;

// Useful for Dijkstra: {distance, node}
pq.push({0, source});
```

Use cases: k largest/smallest elements, Dijkstra, scheduling, merging sorted lists, greedy problems.

8. set and multiset

`set` stores sorted unique values. `multiset` stores sorted values and allows duplicates. Internally, ordered sets are usually balanced binary search trees.

```
set<int> s;
s.insert(3);
s.insert(1);
s.insert(3);  // duplicate ignored

cout << *s.begin();        // smallest
cout << *s.rbegin();       // largest

auto it = s.find(3);
s.erase(3);
```

Operation	set	unordered_set	Notes
insert	O(log n)	Average O(1)	unordered can degrade to O(n)
find/count	O(log n)	Average O(1)	set remains sorted
erase by value	O(log n)	Average O(1)	multiset erases all equal values by value
lower_bound/upper_bound	O(log n)	Not available meaningfully	Ordered containers only

multiset duplicate erase warning

```
multiset<int> ms = {1, 1, 1, 2};
ms.erase(1);              // erases ALL 1s

ms = {1, 1, 1, 2};
ms.erase(ms.find(1));     // erases only one 1
```

9. map, unordered_map, and multimap

`map` stores key-value pairs sorted by key. `unordered_map` uses hashing and gives average O(1) lookup.

```
map<string, int> mp;
mp["apple"] = 3;
mp["banana"]++;

for (auto [key, value] : mp) {
    cout << key << " " << value << "\n";
}

if (mp.find("apple") != mp.end()) {
    cout << "found";
}
```

Operation	map	unordered_map	Notes
insert/update	O(log n)	Average O(1)	map keeps keys sorted
find/count	O(log n)	Average O(1)	Worst case unordered is O(n)
erase	O(log n)	Average O(1)	By key or iterator
iteration order	Sorted by key	Unspecified	Do not rely on unordered order

Frequency counting pattern

```
vector<int> a = {1, 2, 2, 3, 3, 3};
unordered_map<int,int> freq;
for (int x : a) freq[x]++;

for (auto [x, cnt] : freq) {
    cout << x << " appears " << cnt << " times\n";
}
```

Common pitfall: `mp[key]` creates the key if it does not exist. Use `find` when you only want to check presence.

10. Important STL algorithms

Algorithms usually work on iterator ranges. For vectors and arrays, many algorithms are O(n), while sorting is O(n log n).

Algorithm	Syntax	Time
sort	sort(v.begin(), v.end())	O(n log n)
reverse	reverse(v.begin(), v.end())	O(n)
min/max element	*min_element(...), *max_element(...)	O(n)
accumulate	accumulate(v.begin(), v.end(), 0)	O(n)
count	count(v.begin(), v.end(), x)	O(n)
find	find(v.begin(), v.end(), x)	O(n)
binary_search	binary_search(v.begin(), v.end(), x)	O(log n) after sort
lower_bound	lower_bound(v.begin(), v.end(), x)	O(log n) random access
next_permutation	next_permutation(v.begin(), v.end())	O(n) per permutation

Binary search helpers

```
vector<int> v = {1, 2, 4, 4, 4, 7};
int x = 4;

auto lb = lower_bound(v.begin(), v.end(), x); // first >= x
auto ub = upper_bound(v.begin(), v.end(), x); // first > x

int firstIndex = lb - v.begin();
int countX = ub - lb;
```

11. Custom comparators

Comparators define custom ordering. They are used in `sort`, `set`, `map`, and priority queues.

Sort pairs by second value

```
bool comp(pair<int,int> a, pair<int,int> b) {
    if (a.second == b.second) return a.first < b.first;
    return a.second < b.second;
}

vector<pair<int,int>> v = {{1, 5}, {2, 3}, {3, 3}};
sort(v.begin(), v.end(), comp);
```

Lambda comparator

```
sort(v.begin(), v.end(), [](auto &a, auto &b) {
    if (a.second != b.second) return a.second > b.second;
    return a.first < b.first;
});
```

Strict weak ordering: Your comparator should answer “should a come before b?” Do not use invalid logic like `return a <= b` for sorting.

12. Time complexity cheat sheet

Container	Access	Search	Insert	Delete	Sorted?
vector	O(1)	O(n)	Back amortized O(1), middle O(n)	Back O(1), middle O(n)	No
deque	O(1)	O(n)	Ends O(1), middle O(n)	Ends O(1), middle O(n)	No
stack	top O(1)	No search	O(1)	O(1)	No
queue	front O(1)	No search	O(1)	O(1)	No
priority_queue	top O(1)	No search	O(log n)	pop O(log n)	Heap order
set/map	Iterator	O(log n)	O(log n)	O(log n)	Yes
unordered_set/map	Iterator	Avg O(1)	Avg O(1)	Avg O(1)	No
multiset/multimap	Iterator	O(log n)	O(log n)	O(log n)	Yes, duplicates

For `unordered_*` containers, average O(1) assumes a good hash distribution. In adversarial or collision-heavy cases, operations can degrade to O(n).

13. Common DSA patterns with STL

Sliding window maximum

```
// Store indices in decreasing value order
deque<int> dq;
for (int i = 0; i < n; i++) {
    while (!dq.empty() && dq.front() <= i - k) dq.pop_front();
    while (!dq.empty() && a[dq.back()] <= a[i]) dq.pop_back();
```

```
    dq.push_back(i);
    if (i >= k - 1) ans.push_back(a[dq.front()]);
}
```

Top K frequent elements idea

```
unordered_map<int,int> freq;
for (int x : nums) freq[x]++;

priority_queue<pair<int,int>> pq; // {frequency, value}
for (auto [x, f] : freq) pq.push({f, x});

while (k--) {
    cout << pq.top().second << " ";
    pq.pop();
}
```

Two-sum with hash map

```
unordered_map<int,int> pos;
for (int i = 0; i < n; i++) {
    int need = target - a[i];
    if (pos.count(need)) return {pos[need], i};
    pos[a[i]] = i;
}
```

14. Mistakes to avoid

- Using vector erase repeatedly inside a loop: each erase can be O(n), causing O(n²) behavior.
- Forgetting to sort before binary search: `lower_bound` and `binary_search` require sorted ranges.
- Dereferencing end iterator: `v.end()` points after the last element.
- Wrong comparator: comparator should be strict; avoid <= or inconsistent conditions.
- Assuming unordered_map is always O(1): it is average O(1), not guaranteed.
- Erasing all duplicates in multiset accidentally: `ms.erase(value)` removes every matching value.
- Integer overflow in accumulate: use `0LL` as the initial value for long long sums.

```
long long sum = accumulate(v.begin(), v.end(), 0LL);
```

15. Revision checklist

Can you answer this?	Quick answer
Which container gives sorted unique values?	set
Which gives average O(1) lookup?	unordered_set / unordered_map
Which structure is best for BFS?	queue
Which structure gives max/min repeatedly?	priority_queue
What does lower_bound return?	First element >= x
What does upper_bound return?	First element > x
Default priority_queue type?	Max heap
Vector middle erase complexity?	O(n)

Best practice for competitive programming: first solve the logic on paper, then choose the STL container that matches the operations you need most often.

16. One-page memory summary

- `vector`: default choice for arrays; fast random access; sort-friendly.
- `stack`: last-in-first-out; parentheses, DFS, monotonic stack.
- `queue`: first-in-first-out; BFS and level-order traversal.
- `priority_queue`: heap; top element fast; push/pop logarithmic.
- `set/map`: sorted, logarithmic operations.
- `unordered_set/unordered_map`: hashing, average constant-time operations.
- `sort`: O(n log n); `reverse`, `count`, `find`: O(n).
- `lower_bound` and `upper_bound`: binary search tools for sorted ranges.
- Always connect STL syntax to complexity; this is what matters in interviews and contests.